

STRUKTUR DATA

Pertemuan 4

Stack (Tumpukan)

Definisi

Stack atau Tumpukan adalah suatu struktur data yang terbentuk dari barisan hingga yang terurut dari satuan data. Pada Stack, penambahan dan penghapusan elemennya hanya dapat dilakukan pada satu posisi, yaitu posisi akhir stack. Posisi ini disebut Puncak atau Top dari stack.

Stack. . .

Elemen Stack S yang berada pada posisi Top / Puncak dinyatakan dengan :

$$\mathbf{TOP(S)}.$$

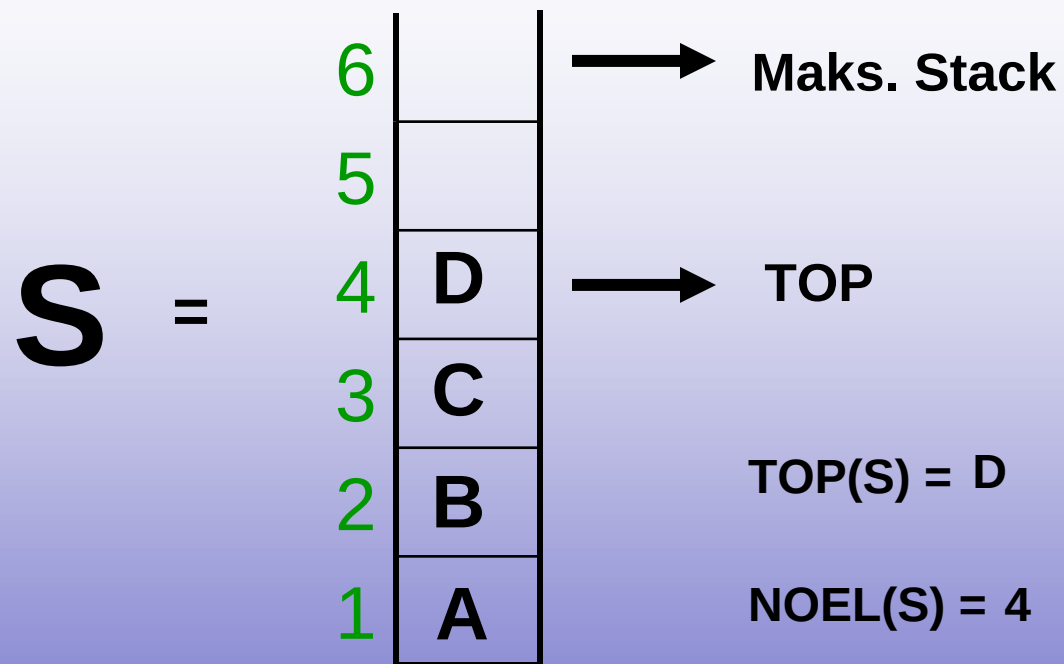
Jika stack $S = [S_1, S_2, \dots, S_T]$ maka :

$$\mathbf{TOP(S) = S_T}$$

Sedang banyaknya elemen stack S dinyatakan dengan :

$$\mathbf{NOEL(S) = T.}$$

Representasi Stack



Operasi Stack

1. CREATE(S)

Adalah operator yang menyebabkan Stack S menjadi suatu stack hampa. Jadi $\text{NOEL}(\text{CREATE}(S))$ adalah 0 dan $\text{TOP}(\text{CREATE}(S))$ adalah tak terdefinisi.

2. ISEMPTY(S)

Adalah operator yang berfungsi untuk memeriksa apakah Stack(S) hampa (kosong) atau tidak. Hasil dari operasi ISEMPTY adalah Boolean yaitu TRUE jika kosong dan FALSE jika isi.

Operasi Stack (lanjutan...

3. PUSH(S,elemen)

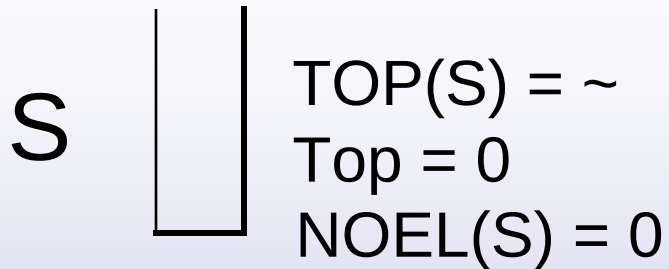
PUSH(S,E) adalah operator yang berfungsi menambahkan elemen E ke Stack. Elemen E akan ditempatkan pada posisi TOP(S). Suatu error akan terjadi jika PUSH dioperasikan pada stack yang sudah mencapai maksimal stack. (*Overflow*)

4. POP(S)

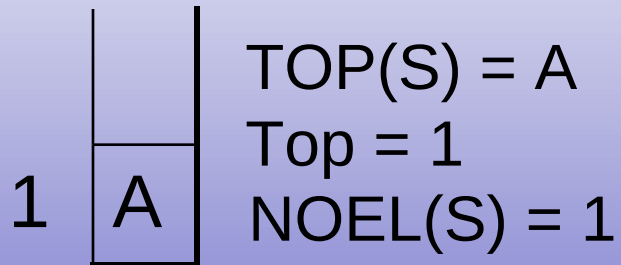
Adalah operator yang berfungsi mengeluarkan atau menghapus elemen TOP(S) dari dalam stack. POP(S) akan mengurangi nilai NOEL(S) dengan 1. Suatu error akan terjadi jika POP(S) dilakukan pada stack yang hampa / kosong. (*Underflow*)

Contoh

CREATE(S)



PUSH(S,'A')



PUSH(S,'K')



POP(S)



Algoritma

1. Algoritma PUSH(Stack,item)

1. IF $\text{MaxStk} = \text{Top}$ THEN Overflow
2. $\text{Top} = \text{Top} + 1$
3. $\text{STACK}[\text{Top}] = \text{item}$
4. Exit

2. Algoritma POP(Stack)

1. IF $\text{Top} = 0$ THEN Underflow; Exit
2. $\text{Item} = \text{Stack}[\text{Top}]$
3. $\text{Top} = \text{Top} - 1$
4. Exit

TUGAS 1

1. CREATE(S)
2. PUSH(S,'K')
3. IF NOEL(S)>5 THEN GOTO 6
4. PUSH(S,'E')
5. GOTO 2
6. IF ISEEMPTY(S) THEN GOTO 9
7. PRINT TOP(S)
8. PRINT NOEL(S)
9. END

Gambarkan keadaan Stack S selama proses diatas berlangsung.

TUGAS 2

```
CREATE(S)
WHILE NOEL(S) <= 5 DO
    PUSH(S,item)
    PRINT TOP(S)
    PRINT NOEL(S)
ENDWHILE
REPEAT
    POP(S)
    IF NOEL(S)=2 THEN PRINT TOP(S)
UNTIL NOEL(S)=1
PRINT TOP(S)
PRINT NOEL(S)
PRINT ISEMPY(S)
END
```

Jelaskan hasil dari algoritma diatas, lengkapi dengan hasil output yang mungkin muncul.

APLIKASI STACK

Aplikasi stack bisa diterapkan dalam bidang aritmatika yaitu dalam penulisan ekspresi matematika. Dalam penulisannya harus memperhatikan hierarki Operator yaitu :

1. (...)
2. ^ (pangkat)
3. /, *, DIV, MOD
4. +, -

Operator Logika :

1. NOT
2. AND
3. OR

Aplikasi stack...

Bentuk notasi penulisan ekspresi matematika

- | | | | |
|----|------------------|-----------|------------------------------|
| 1. | Infix Notation | → $A + B$ | → Operand, Operator, Operand |
| 2. | Prefix Notation | → $+ A B$ | → Operator, Operand, Operand |
| 3. | Postfix Notation | → $A B +$ | → Operand, Operand, Operator |

Contoh :

Infix → $(4 - 3) * (12 / 3)$

Prefix → $*, -, 4, 3, /, 12, 3$

Postfix → $4, 3, -, 12, 3, /, *$

Contoh Soal :

1. Tentukan hasil dari : $12, 7, 3, -, /, 2, 1, 5, +, *, +$
2. $5 + 3^2 - 8 / 4 + 3 + 6$
3. $3, 1, + 2, ^, 7, 4, -, 2, *, +, 5, -$
4. $(A * ((B + D) / E)) - (F * (G + (H / K)))$

Algoritma Postfix

Untuk membuat notasi infix ke postfix, berikut ini adalah algoritmanya:

Algoritma PostFix(Q, P) {Q=Infix, P=Postfix}

1. Masukkan '(' ke dalam Stack dan ')' ke akhir infix.
2. Telusuri Q sampai elemen yang terakhir
3. Jika menemukan *Operand*, maka masukkan ke P.
4. Jika menemukan '(', maka masukkan ke dalam Stack.
5. Jika menemukan operator, maka :
 1. Ulangi POP(S) setiap Operator yang sama atau lebih tinggi hirarkinya dan masukkan ke P
 2. Masukkan operator tadi ke Stack.
6. Jika menemukan ')', maka :
 1. Ulangi POP(S) setiap *Operator* sampai menemukan '(' dan masukkan ke dalam P.
 2. Hapus '('
7. Selesai

Penerapan Algoritma Postfix

$$Q = A + (B * C - (D / E \wedge F) * G) * H$$

	Q	STACK	POSTFIX
		(	
1	A	(	A
2	+	(+	A
3	(	(+(	A
4	B	(+(	AB
5	*	(+(*	AB
6	C	(+(*	ABC
7	-	(+(-	ABC*
8	(	(+(-(	ABC*
9	D	(+(-(	ABC*D

	Q	STACK	POSTFIX
10	/	(+(-(/	ABC*D
11	E	(+(-(/	ABC*DE
12	^	(+(-(/^	ABC*DE
13	F	(+(-(/^	ABC*DEF
14	)	(+(-	ABC*DEF^/
15	*	(+(-*	ABC*DEF^/
16	G	(+(-*	ABC*DEF^/G
17	)	(+	ABC*DEF^/G*-
18	*	(+*	ABC*DEF^/G*-
19	H	(+*	ABC*DEF^/G*-H
20	)	...	ABC*DEF^/G*-H*+

Contoh Aplikasi Menara Hanoi

Contoh : Tower Menara Hanoi

Tujuan : Memindahkan lempengan di A ke C dengan bantuan B.

Aturan :

1. Pemindahan = POP pada stack yaitu 1 elemen sekali POP.
2. Yang terkecil selalu diatas yang besar.

Ditanya :

1. Berapa langkah untuk memindahkan lempengan tersebut.

1. Rumus mencari langkah :

$$J = 2^n - 1$$

n = Banyak Lempengan

	Q	STACK	POSTFIX
		(	
1	A	(	A
2	+	(+	A
3	(	(+(	A
4	B	(+(	AB
5	*	(+(*	AB
6	C	(+(*	ABC
7	-	(+(-	ABC*
8	(	(+(-(	ABC*
9	D	(+(-(	ABC*D

	Q	STACK	POSTFIX
10	/	(+(-(/	ABC*D
11	E	(+(-(/	ABC*DE
12	^	(+(-(/^	ABC*DE
13	F	(+(-(/^	ABC*DEF
14	)	(+(-	ABC*DEF^/
15	*	(+(-*	ABC*DEF^/
16	G	(+(-*	ABC*DEF^/G
17	)	(+	ABC*DEF^/G*-
18	*	(+*	ABC*DEF^/G*-
19	H	(+*	ABC*DEF^/G*-H
20	)	HABIS	ABC*DEF^/G*-H*+

ABC-^D/E*F+

1. A^B-C/D^*E+F
2. A^B-C/D^*E+F
3. $A^B/C-D^*E+F$
4. $A(B-C)^D/E^*F+$
5. $(A^B(C-D))^E/F^*G+$
6. $((A^B(C-D))/D)E^*F+$
7. $A^B(C-D)/D^*E+F$
8. A^B-BC/D^*E+F
9. A^B-BC/D^*E+F
10. $/^B A-BCD^*E+F$
11. $+^B /^B A-BCDEF$

1. $+^B /^B A-BCDEF$
2. $^B - /^B + ABCDEF$
3. $+^B /^B AB-CDEF$

Tugas 1

$$(A * ((B + D) / E)) - (F * (G + (H / K)))$$

Ubah ke dalam notasi Postfix dengan menggunakan Algoritma Postfix.

Tugas 2

1. Ubah Notasi infix ini ke prefix dan postfix
 - a. $A/B-C/D$
 - b. $(A+B)^3-C*D$
 - c. $A^{(B+C)}$
2. Ubah notasi Prefix ini ke dalam infix dan postfix
 - a. $+/-ABC^DE$
 - b. $-+DE/XY$
 - c. ^+23-CD
3. Ubah notasi Postfix ini ke dalam infix dan prefix
 - a. $ABC+-$
 - b. $GH+IJ/*$
 - c. AB^CD+-